

LECTURE

Understanding the Terraform Blocks



Terraform Blocks

The Building Blocks of Infrastructure



Clear Structure

Each component has a specific purpose, making it easy to organization and understand your configurations



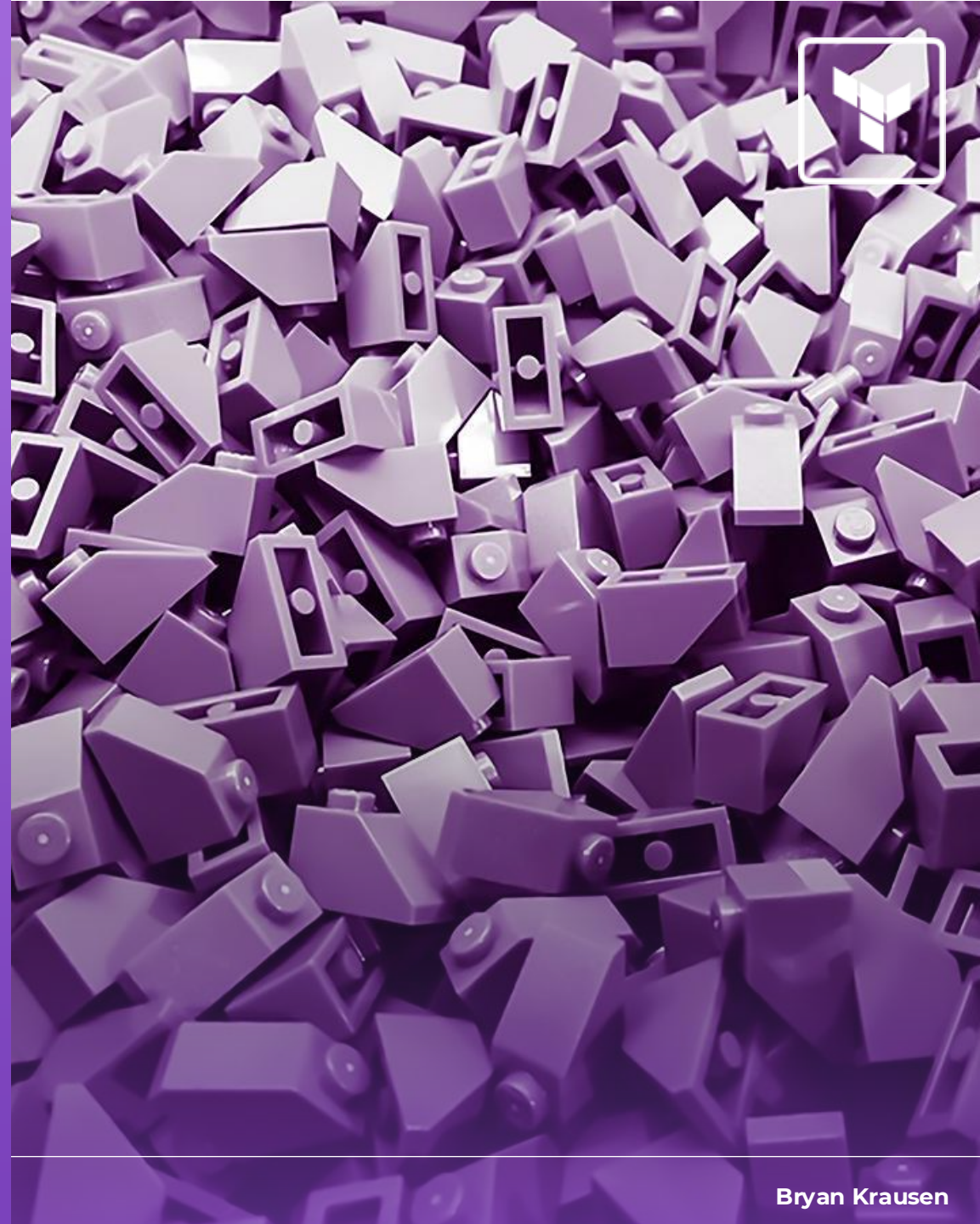
Flexible and Modular

Terraform is designed to be flexible. You don't need to use all the components to build infrastructure



Reusable Patterns

Using components thoughtfully helps create reusable, modular configurations that are easy to maintain and scale





Terraform Block Types



provider

Connect Terraform to cloud platforms or services (e.g., AWS, Azure, GCP, VMware)



resource

The infrastructure that Terraform creates, updates, or deletes (e.g., VMs, storage)



data

Retrieve information about existing resources to use in your configuration



variable

Make code reusable by defining values that can be customized for each run



output

Display or share essential information about resources after deployment



terraform

Define specific settings or resources in a configuration, like req. Terraform version



module

Reusable configurations that group related resources – makes it easy to share configs



import

Pull in existing resources into Terraform management

LECTURE

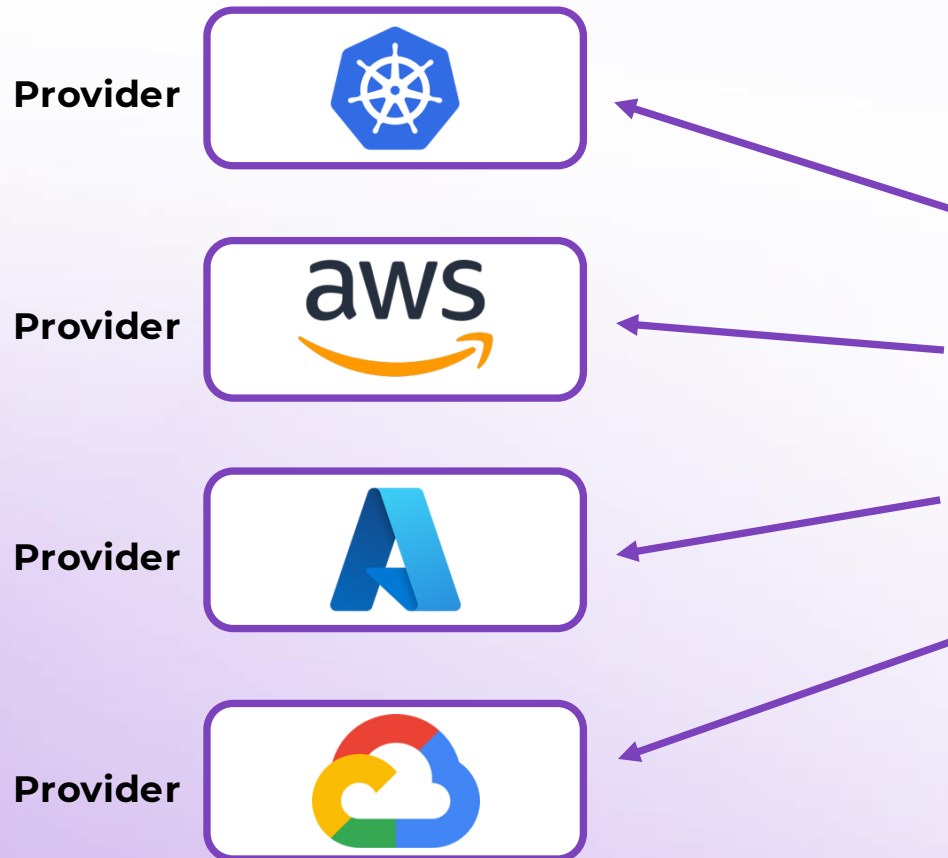
Provider Block



Terraform Providers

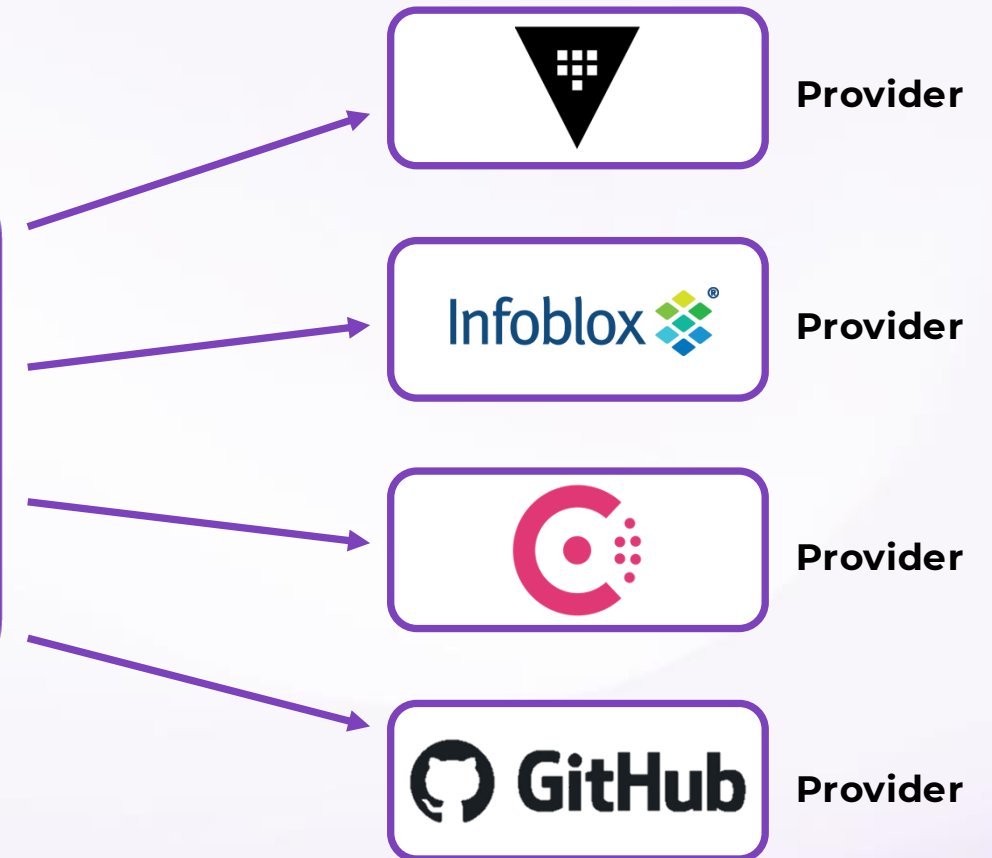


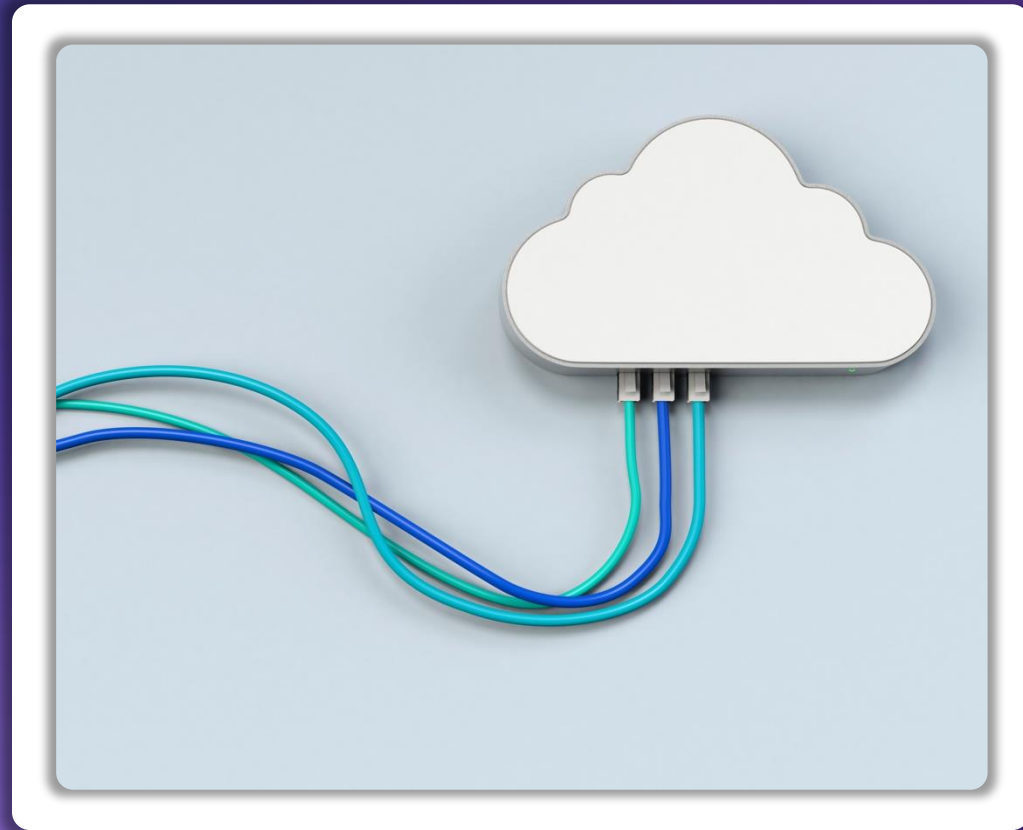
Cloud Platforms



Terraform Core

Other Services





Provider Block

The provider **block** is used to extend Terraform's functionality. It allows you to connect Terraform to any API-driven platform, such as cloud platforms, SaaS applications, on-premises hardware solutions, and more.

Provider blocks define where and what Terraform can manage, such as AWS for cloud resources or GitHub for repositories.



Provider Block

Plugins that extend the functionality of Terraform and enable it to interact with cloud providers, SaaS providers, and other API-supported apps.

- **Resources are Implemented by a Provider**

Without providers, Terraform would not be able to provision or manage any infrastructure.

- **Who Publishes Providers?**

HashiCorp maintains some providers, but some are managed by partners and the Terraform community.

- **Developed Separately From Terraform Core**

This means functionality can be added/changed per version.

```
provider "aws" {  
    region  = "us-east-2"  
    profile = "prd-workload"  
}  
  
provider "azurerm" {  
    features {}  
  
    tenant_id           = "tenant-id"  
    subscription_id    = "sub-id"  
    client_id           = "client-id"  
    client_secret       = "client-secret"  
}
```

Breaking Down the Provider Block



```
providers.tf

provider "aws" {
  region = "us-east-2"
  profile = "prd-workload"
}

provider "azurerm" {
  features {}

  tenant_id       = "tenant-id"
  subscription_id = "sub-id"
  client_id       = "client-id"
  client_secret   = "secret"
}
```

The name of the provider defined by the developer or maintainer

Arguments specific to the provider – found in documentation

Authentication arguments – should be provided via Environment Variables instead due to security concerns

® Copyright Bryan Krausen – DO NOT DISTRIBUTE

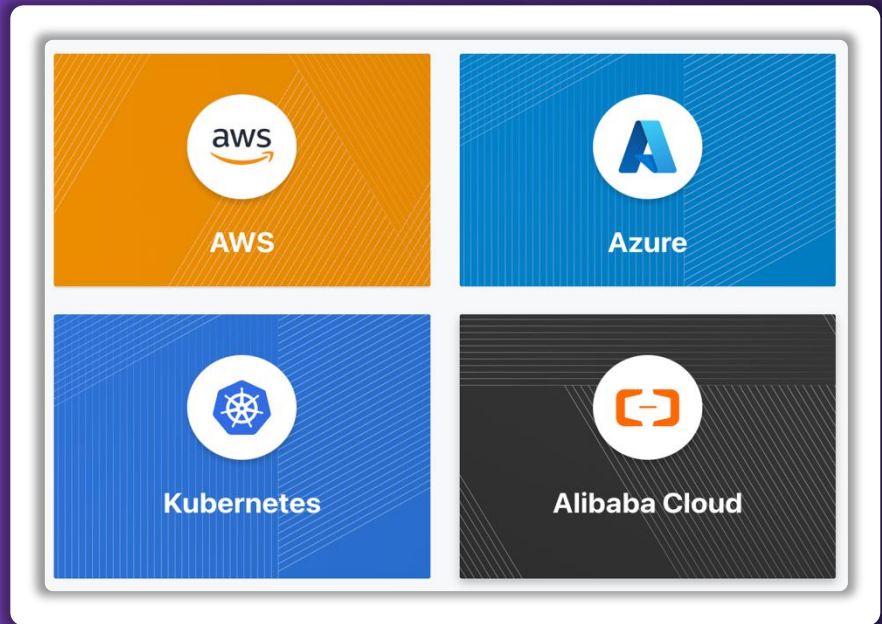
Provider Block



- ✧ Some providers need credentials or access tokens to authenticate and manage resources on the target platform (e.g., AWS access keys, GitHub API token)
- ✧ A single provider block enables Terraform to manage multiple resources on that platform (e.g., a single AWS provider enables EC2 instances, S3 buckets, EKS clusters)



Provider Block



- ❧ While providers are stored in GitHub, the documentation can be found on the Terraform Registry → registry.terraform.io
- ❧ When declared in your configuration, the provider is downloaded to the machine executing Terraform

provider Meta-Argument

Explicitly Assign Resources to a Specific Provider



- ✚ Explicitly assigns a resource to a specific provider configuration.
- ✚ Most often used for multi-cloud or multi-account deployments.
- ✚ Use it when deploying resources across multiple cloud accounts or regions or managing various instances of the same provider (e.g., *separate AWS accounts for dev and prod*).
- ✚ Uses the `provider = <provider_alias>` syntax

```
1  provider "aws" {
2    region = "us-east-1"
3  }
4
5  provider "aws" {
6    alias  = "prod"
7    region = "us-west-1"
8  }
9
10 resource "aws_s3_bucket" "dev_bucket" {
11   bucket = "my-dev-bucket"
12 }
13
14 resource "aws_s3_bucket" "prod_bucket" {
15   provider = aws.prod
16   bucket   = "my-prod-bucket"
17 }
```

LECTURE

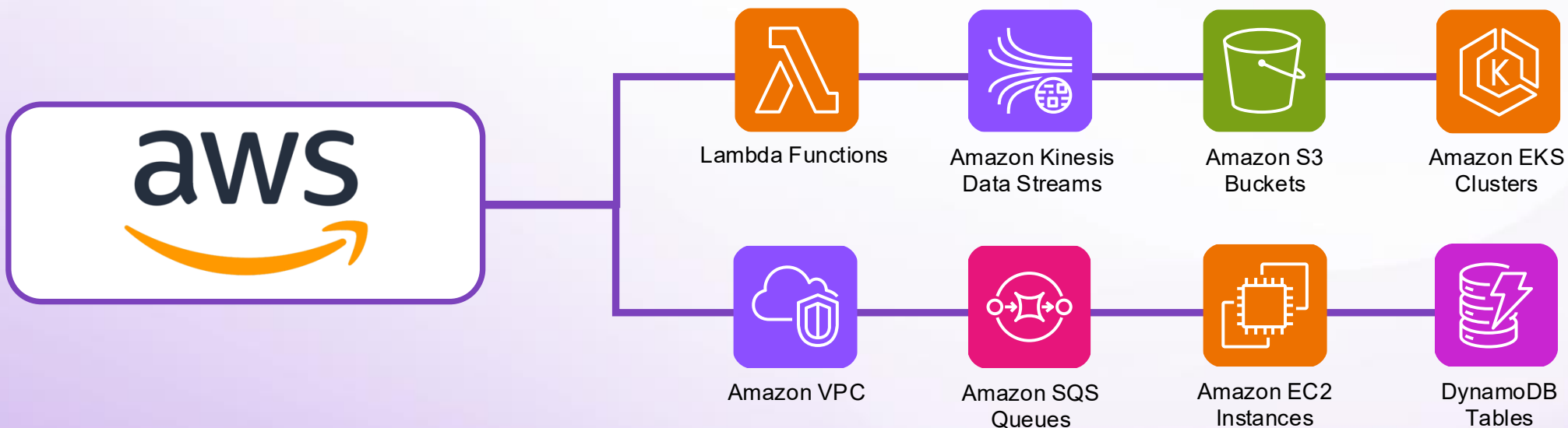
Resource Block



Terraform Resources



Terraform Resources





Resource Block

The resource block is the core element in Terraform. It defines the specific infrastructure pieces you want to create, manage, or modify, such as virtual machines, databases, or networks.

Resource blocks use a resource type (e.g., `aws_instance`) and a unique name. It acts as a blueprint with the settings you need to configure each resource.



Resource Block

A resource block tells Terraform what to build and how it should be configured.



What Type of Resource is Needed?

The resource type identifies the type of resource that you need. Value is determined by the provider depending on the desired resource.



How Should the Resource be Configured?

Each resource block includes settings (or arguments) that configure its behavior.



Resource Identifiers

Each resource block is identified by a **type** and a **name**, which makes it easy to reference in other parts of your configuration

```
resource "aws_instance" "web" {  
    ami           = "ami-012345"  
    instance_type = "t2.micro"  
  
    key_name    = "prd-web-key"  
    subnet_id  = "subnet_12345abc"  
  
    tags = {  
        Name = "prd-web-svr_01"  
    }  
}  
  
resource "aws_instance" "db" {  
    instance_type = "m5.4xlarge"  
    ami           = "ami-0c55b159"  
}
```



Breaking Down the Resource Block

```
resource "aws_instance" "web" {  
  ami           = "ami-012345"  
  instance_type = "t2.micro"  
  
  key_name    = "prd-web-key"  
  subnet_id  = "subnet_12345abc"  
  
  tags = {  
    Name = "prd-web-svr_01"  
  }  
}  
  
resource "aws_instance" "db" {  
  instance_type = "m5.4xlarge"  
  ami           = "ami-0c55b159"  
}
```

The type of resource to be created/
managed - defined within the provider

The resource name – user-provided string
- must be unique per configuration.

Arguments specific to the resource type
Can be found in provider documentation

A second resource block within the
same Terraform configuration file

Resource Block



Resource Arguments

```
resource "aws_instance" "web" {  
  ami          = "ami-012345"  
  instance_type = "t2.micro"  
  
  key_name  = "prd-web-key"  
  subnet_id = "subnet_12345abc"  
  
  tags = {  
    Name = "prd-web-svr_01"  
  }  
}  
  
resource "aws_instance" "db" {  
  instance_type = "m5.4xlarge"  
  ami          = "ami-0c55b159"  
}
```

The Image to Use

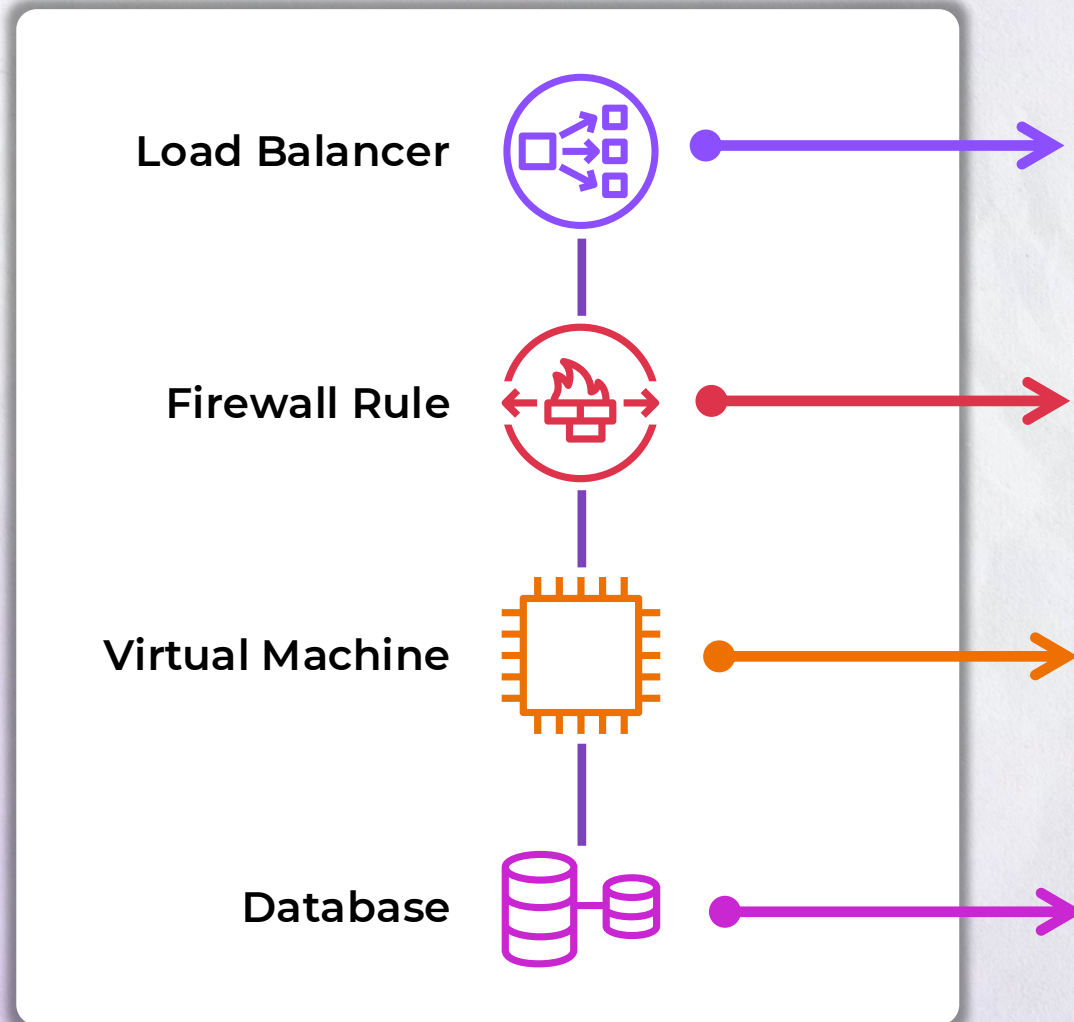
The Size of the Instance

Key Name for SSH Access

The Subnet used to Provision the Instance

Add Tags for the Resource

Resource Block



```
web_app.tf

resource "aws_lb" "public" {
  name = "prd-web-lb"
  load_balancer_type = "network"
}

resource "fortios_firewall_policy" "db" {
  action = "accept"
  name   = "allow_web_443"
}

resource "aws_instance" "web" {
  instance_type = "t3.large"
  ami           = "ami-0c55b159"
}

resource "aws_db_instance" "prd-db" {
  engine           = "mysql"
  instance_class   = "db.t3.large"
}
```



Resource Referencing

Dynamically pull information from one resource block to another, creating a connected configuration

Reference a Resource block by using the resource **type** and the **name** of the resource.

1. `aws_lb.public_load_balancer`
2. `fortios_firewall_policy.db`
3. `aws_instance.web`
4. `aws_db_instance.prdb`

1

```
resource "aws_lb" "public_load_balancer" {  
  name = "prd-web-lb"  
  load_balancer_type = "network"  
}
```

2

```
resource "fortios_firewall_policy" "db" {  
  action = "accept"  
  name   = "allow_web_443"  
}
```

3

```
resource "aws_instance" "web" {  
  instance_type = "t3.large"  
  ami           = "ami-0c55b159"  
}
```

4

```
resource "aws_db_instance" "prdb" {  
  engine           = "mysql"  
  instance_class   = "db.t3.large"  
}
```

Resource Block



```
main.tf

resource "github_repository" "prod_repo" {
  name          = "prod-app-xyz-repo"
  visibility    = "private"
}

resource "github_branch" "default" {
  repository = github_repository.prod_repo.name
  branch     = "main"
}

resource "github_branch_default" "default" {
  repository = github_repository.prod_repo.name
  branch     = github_branch.default.branch
}
```

Resource Block



- ✦ Resource names must start with a letter or underscore, and may contain only letters, digits, underscores, and dashes.

LECTURE

Data Block



Accessing Existing Resources

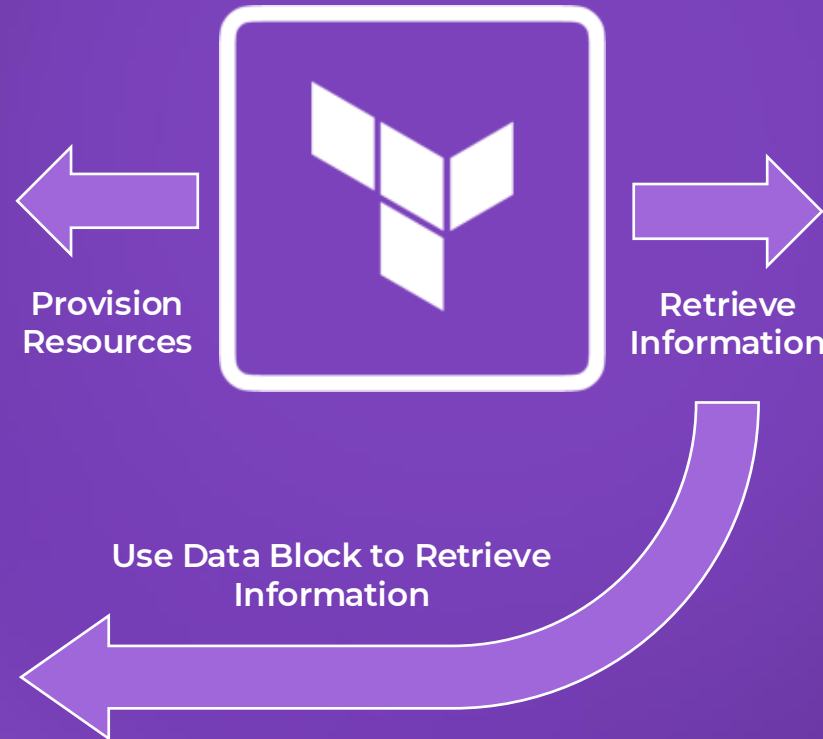


```
resource "azurerm_resource_group" "prd" {
  name      = "example-resources"
  location  = "West Europe"
}

resource "azurerm_virtual_network" "dv" {
  name                = "example-network"
  resource_group_name = "azure"
  location            = "west"
  address_space       = ["10.0.0.0/16"]
}

resource "azurerm_mssql_database" "db1" {
  name         = "example-db"
  server_id    = "server_db"
  collation    = "sql_general"
}

data "azurerm_kubernetes_cluster" "aks" {
  name                = "prod-cluster"
  resource_group_name = "prod-group"
}
```



Existing Resources



Virtual Machine



Kubernetes Service



CosmosDB



Storage Account



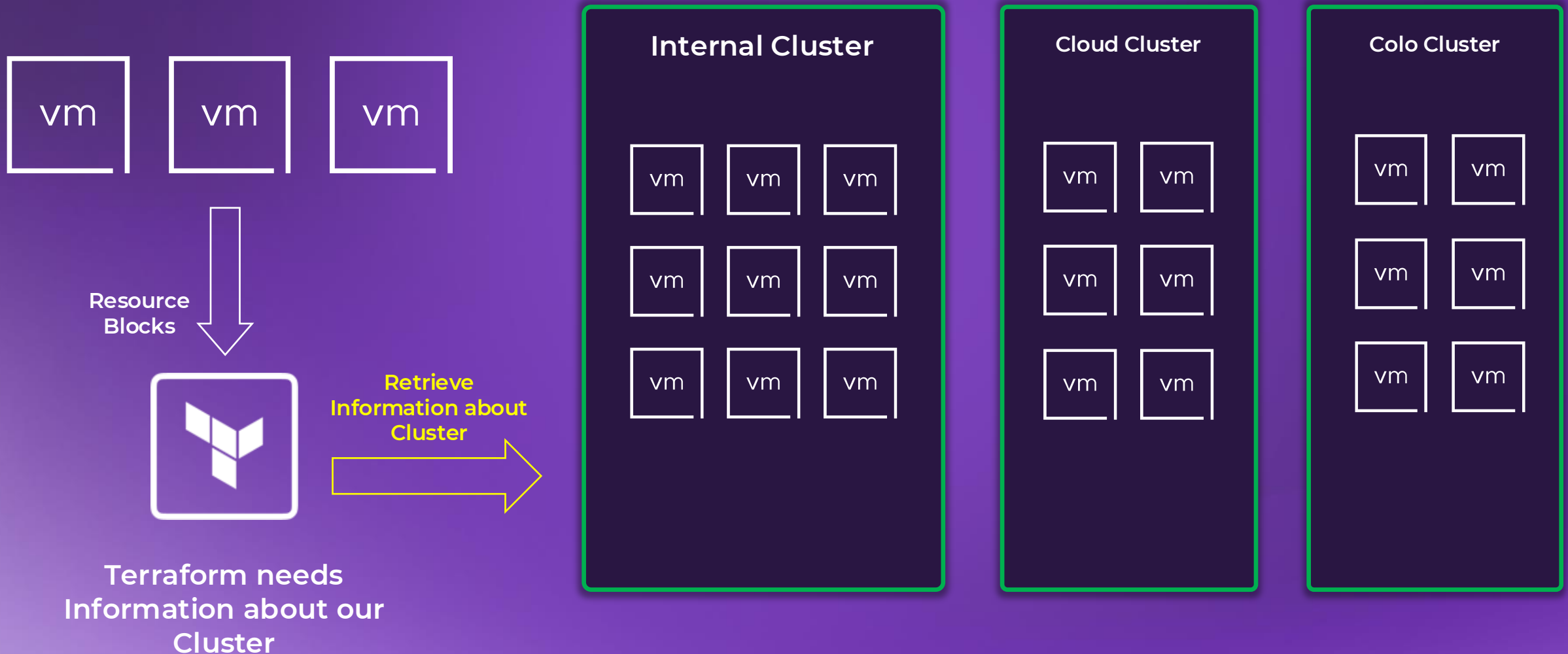
Image Template



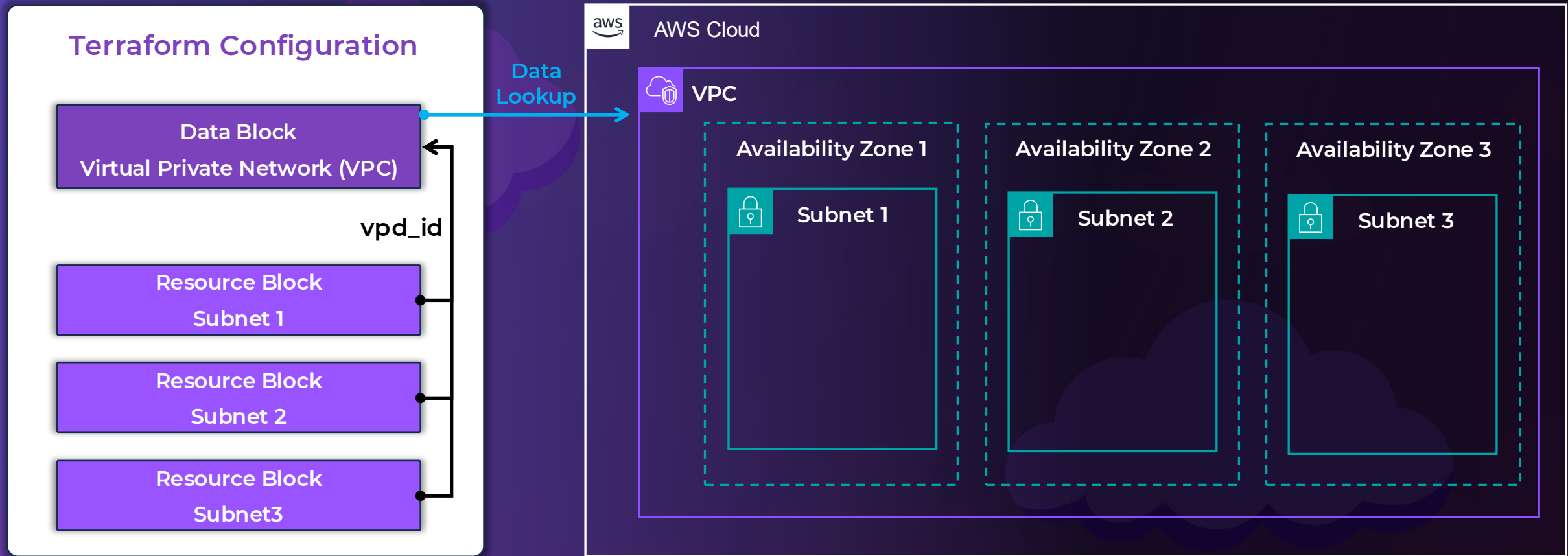
Entra Domain



Existing Resources - Example



Existing Resources - Example





Data Block

A **data** block retrieves information about existing resources or external data without creating new infrastructure. It allows you to dynamically reference attributes, such as the ID of an existing subnet or the details of a Kubernetes cluster, making configurations flexible and reusable.

By using **data** blocks, you can integrate Terraform with resources managed outside of its configuration.





Data Block

Data blocks allow Terraform to access information about existing resources rather than creating new ones.



Access Existing Infrastructure

Retrieve the details of existing resources so they can be referenced in other Terraform blocks



Dependency Resolution

Data blocks help when one resource relies on the properties of another existing resource



Avoid Hardcoding Values

Data blocks prevent the need for hardcoded values by dynamically pulling in details, which makes configurations more adaptable

```
network.tf

data "aws_vpc" "prd" {
  filter {
    name     = "tag:Name"
    values   = ["prd-vpc"]
  }
}

resource "aws_subnet" "pub" {
  vpc_id      = data.aws_vpc.prd.id
  cidr_block  = "10.0.6.0/24"
  ...
}

output "prd_vpc_subnet" {
  value = data.aws_vpc.prd.cidr_block
}
```

Breaking Down the Data Block

The given name used to reference the data block – user-defined string



The type of the resource to obtain data about - defined by provider

Query constraints defined by the data source to get the data you want

Get data about an Azure Resource Group named `dev-resource-group`

Get data about the default namespace for a Kubernetes cluster

```
main.tf

data "aws_vpc" "prd" {
  filter {
    name     = "tag:Name"
    values   = ["prd-vpc"]
  }
}

data "azurerm_resource_group" "dev" {
  name = "dev-resource-group"
}

data "kubernetes_namespace" "app" {
  metadata {
    name = "customer-app"
  }
}
```



Referencing Data Blocks

Get information from a data block to use elsewhere within the configuration.

data blocks retrieve a wide range of information about the specified resource – called **attributes**.

Data block attributes are referenced just like resource attributes: `data.<type>.<name>.<attribute>`

- o `data.aws_vpc.prd.arn`
- o `data.aws_vpc.prd.cidr_block`
- o `data.aws_vpc.prd.id`
- o `data.aws_vpc.prd.owner_id`

```
network.tf

data "aws_vpc" "prd" {
  filter {
    name     = "tag:Name"
    values   = ["prd-vpc"]
  }

  "arn": "arn:aws:ec2:us-west-2:xxx:vpc/vpc-0ab"
  "cidr_block": "10.0.0.0/16"
  "default": false,
  "dhcp_options_id": "dopt-0abcdef1234567890",
  "enable_dns_hostnames": true,
  "enable_dns_support": true,
  "id": "vpc-0abcd1234efgh5678",
  "instance_tenancy": "default",
  "ipv6_association_id": null,
  "ipv6_cidr_block": null,
  "main_route_table_id": "rtb-0abcdef1234567890",
  "owner_id": "123456789012",
  "tags": {
    "Name": "prd-vpc"
  }
}
```

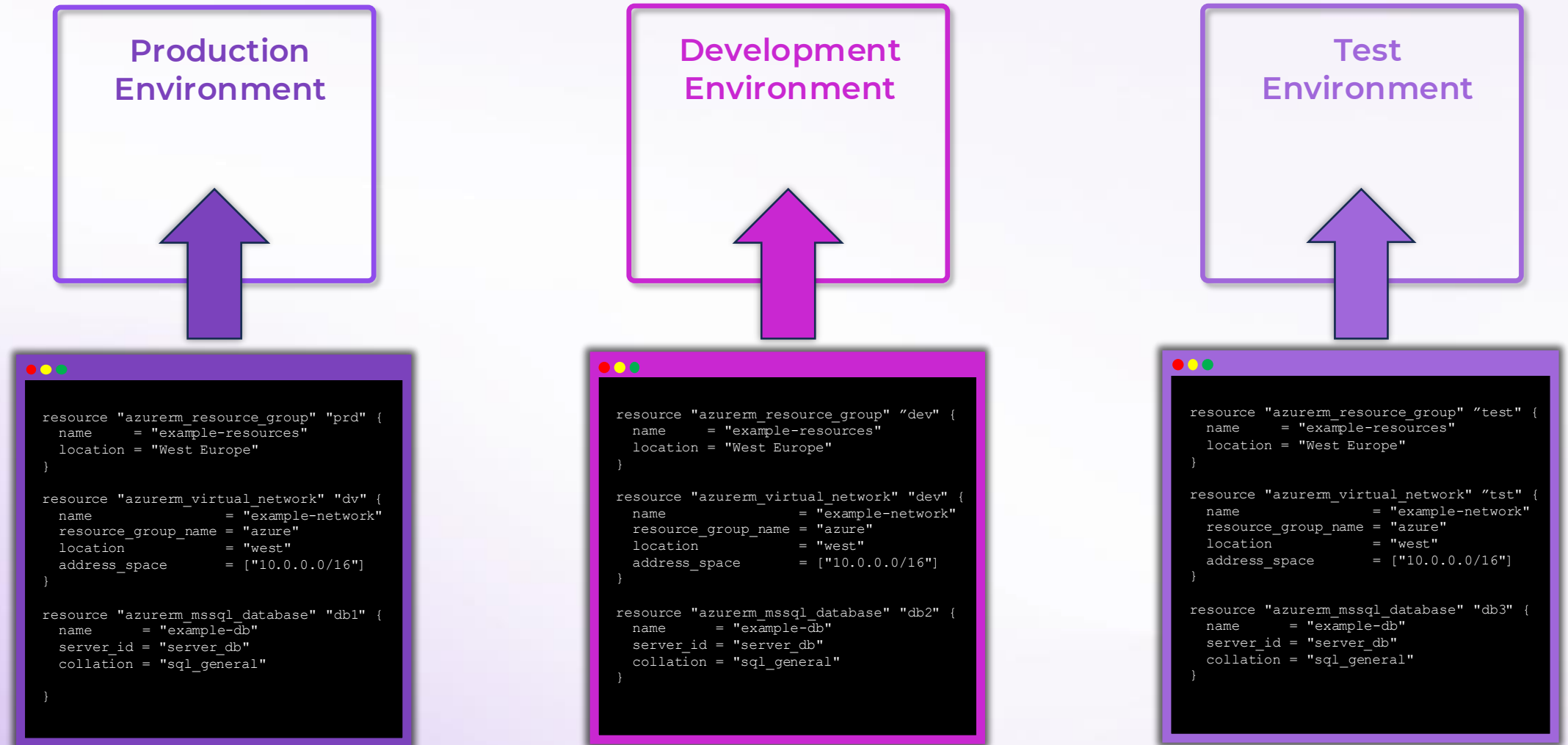
This data block gets all this data

LECTURE

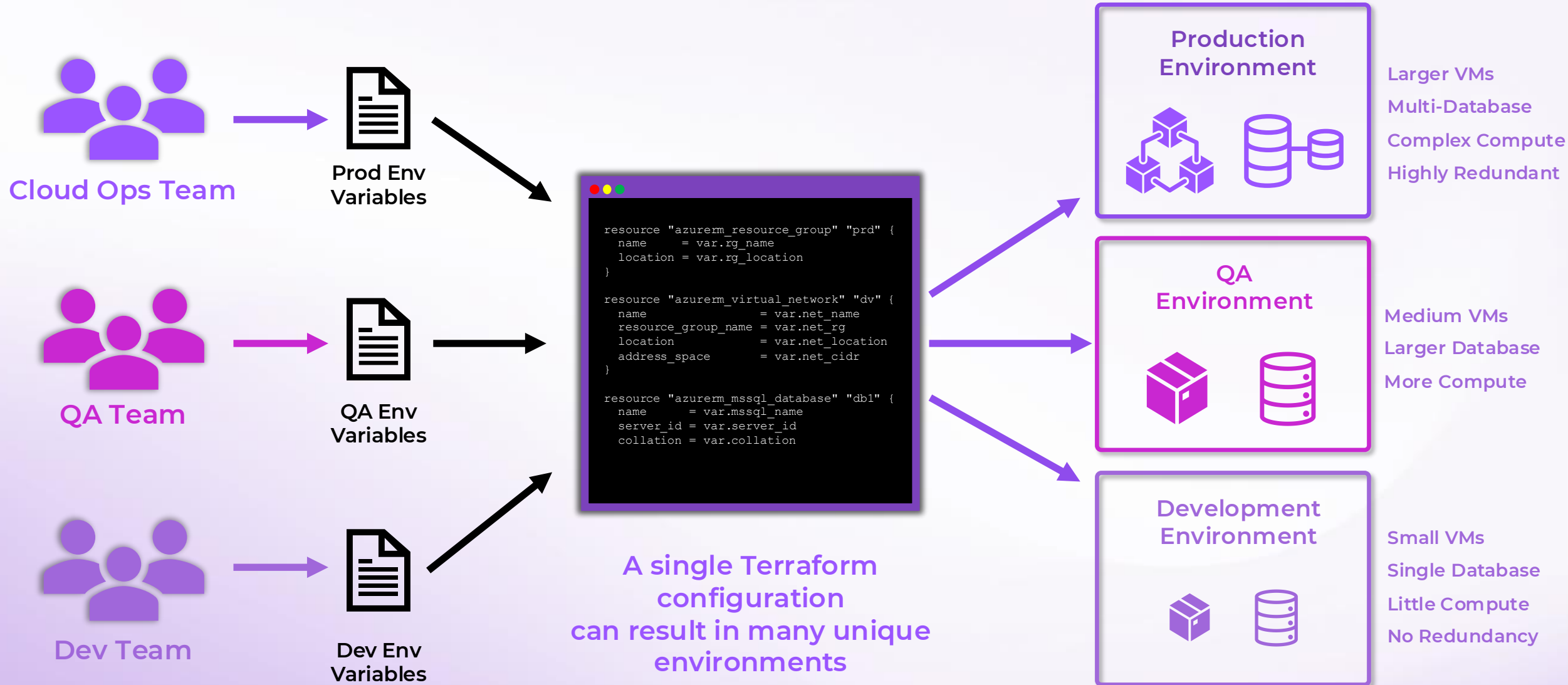
Variable Block

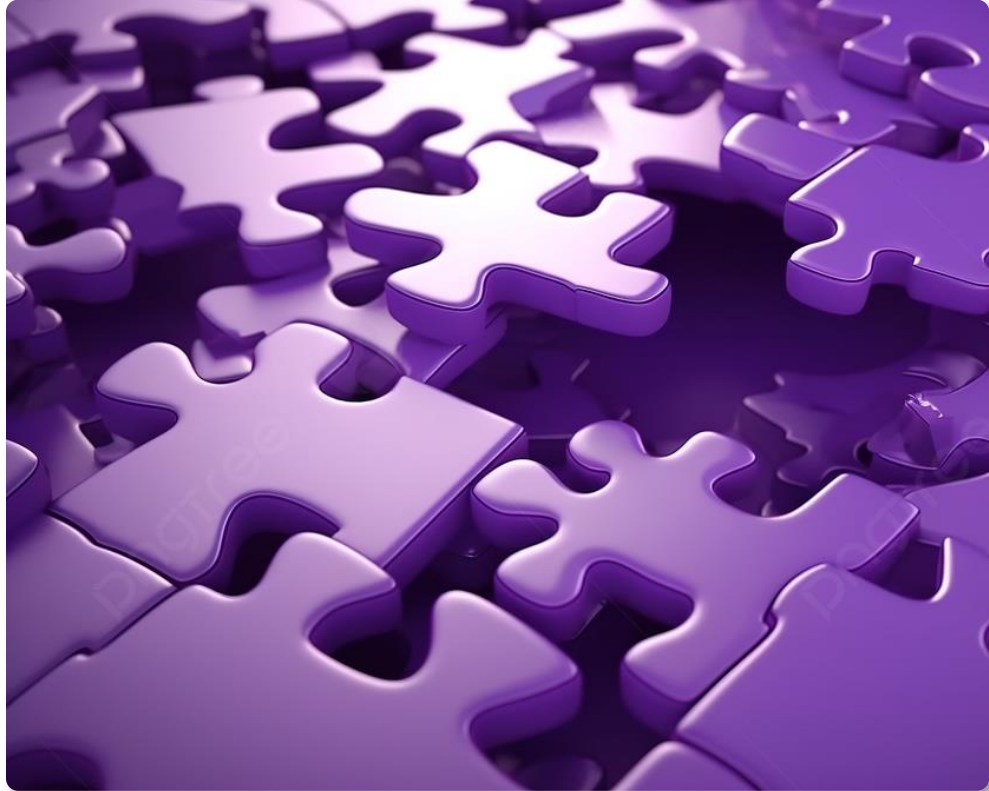


The Problem: Hardcoding Values in Terraform



The Solution: Using Variables in Terraform





Variable Block

A **variable** block is used to define inputs that can be reused and customized, making your configurations more flexible and dynamic. Variables allow you to parameterize values instead of hardcoding them directly into resource blocks.

This approach improves reusability, simplifies managing multiple environments, and helps maintain cleaner, more maintainable configurations.



Variable Block

Define an input that can be reused and customized to make configurations more flexible and dynamic



Dynamic Inputs for Dynamic Code

Variables allow configurations to adapt by using different values for different environments or scenarios.



Centralized Management of Values

By defining variables in one place, you can update values easily without modifying multiple resource blocks.



Improved Reusability of Code

Variable blocks make Terraform configurations modular and reusable across projects and teams.

```
variable "vsphere_datacenter" {
  description = "Name of datacenter"
  type        = string
  default     = "prd-workload-dc"
}

variable "vsphere_networks" {
  description = "List of networks"
  type        = list(string)

  default = [
    "VM Network",
    "Management Network"
  ]
}
```

Breaking Down the Variable Block



```
variable 'vsphere datacenter' {  
  description = "Name of datacenter"  
  type = string  
  default = "prd-workload-dc"  
}  
  
variable "vsphere_networks" {  
  description = "List of networks"  
  type        = list(string)  
  
  default = [  
    "VM Network",  
    "Management Network"  
  ]  
}
```

• The name for the variable – a user-provided string

• Friendly description of the variable and what it's used for...

• Define what value types are accepted for the variable

• The default value of the variable if not set anywhere else in Terraform

Types of Variables

Primitive Variable Types



ABC

String

a sequence of characters
representing some text



```
1 variable "region" {  
2     type    = string  
3     default = "us-east-2"  
4 }
```

9

Number

a numeric value



```
1 variable "num_of_vms" {  
2     type    = number  
3     default = 3  
4 }
```

T/F

Bool

a boolean value – either True
or False



```
1 variable "enable_ha" {  
2     type    = bool  
3     default = true  
4 }
```

Types of Variables

Complex Variable Types



List

A sequence of similar values

Presented by a pair of square brackets `[]` containing a comma-separated sequence of values

Note: lists start at 0

```
1 variable "permitted_size" {  
2     type = list  
3     default = ["t3.small", "t4g.micro"]  
4 }
```

index = 0

index = 1

Types of Variables

Complex Variable Types



Map

A group of values identified by named labels

Presented by a pair of curly braces { } and contains a series of key=value pairs

```
1 variable "course_details" {
2   description = "Course details"
3   type        = map(string)
4   default     = {
5     instructor = "bryan"
6     course     = "terraform"
7   }
8 }
```

var.course_details.instructor

var.course_details.course

Types of Variables

Complex Variable Types



Set

A collection of guaranteed unique values

Terraform does not support accessing elements of a set using an index because sets are unordered collections.

Note: Convert to a list to access



```
1 variable "pub_subnet_ids" {  
2     type      = set(string)  
3     default = ["subnet-12345", "subnet-67890"]  
4 }
```



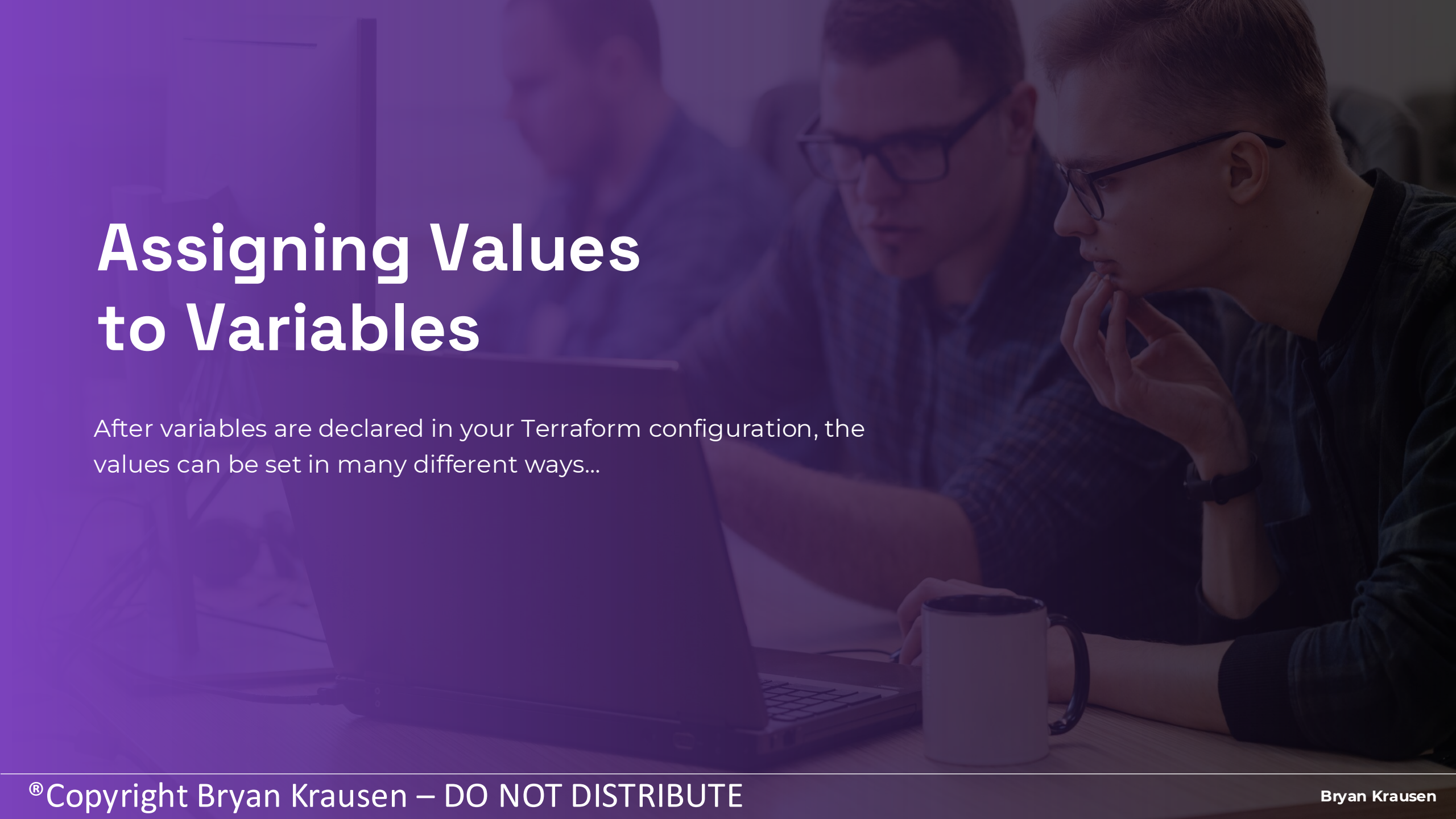
Resource Referencing

Reference a Variable within the Terraform configuration to use the value

Reference a Variable by using the **name** of the variable.

1. `var.vsphere_datacenter`
2. `var.vsphere_networks[0]`

```
variable "vsphere_datacenter" {  
  1 description = "Name of datacenter"  
  type        = string  
  default     = "prd-workload-dc"  
}  
  
2 variable "vsphere_networks" {  
  description = "List of networks"  
  type        = list(string)  
  
  default = [  
    "VM Network",  
    "Management Network"  
  ]  
}
```



Assigning Values to Variables

After variables are declared in your Terraform configuration, the values can be set in many different ways...

Variable Defaults



Set a value directly in the variable block declaration

```
1 variable "pub_subnet_ids" {  
2   type    = set(string)  
3   default = ["subnet-12345", "subnet-67890"]  
4 }
```

Environment Variables



- ✧ Use an environment variable to set the value
- ✧ Must start with `TF_VAR_` followed by the variable name
- ✧ Useful for when you don't want to commit the value to a repo

```
$ export TF_VAR_vsphere_network="10.0.5.0/24"
$ export TF_VAR_vm_image="image-x3f83j2sv3"
$ export TF_VAR_enable_logging=true

PS> $env:TF_VAR_enabled_logging="true"
PS> $env:TF_VAR_subscription_id="abcd-1234-cc"
```



tfvars File



- ❧ Use a `.tfvars` file to set variable values
- ❧ Terraform automatically loads a `.tfvars` file if they exist
- ❧ Set values using a `key=value` format using the variable names
- ❧ A `.tfvars` file is usually excluded from version control (`.gitignore`)

```
terraform.tfvars

# VM-level configurations
vsphere_network = "10.0.5.0/24"
vm_image        = "image-x3f83j2sv3"

# Application Configurations
enable_logging = true

# Cloud Account Configurations
subscription_id = "abcd-1234-cc"
```

Command Line Flags



- ✧ Pass variable values directly from the command line.
- ✧ Use `-var="key=value"` to set the value for a specific variable.
- ✧ Command-line `-var` values override both default values and values in `.tfvars` files.
- ✧ Useful for quick, temporary changes or testing without modifying files.

TERMINAL

```
$ terraform plan -var="enable_logging=true"

$ terraform apply -var="region=us-west-2" -var="vm_image=image-x3f83j2sv3"
```



Order of Precedence

What hierarchy does Terraform follow when resolving variable values from different sources?



Flexibility for Values

Understanding the order of precedence ensures the right value is applied



Clear Override Rules

Higher-priority sources override lower-priority defaults, preventing unexpected behavior.



Adaptability Across Environments

Precedence allows you to manage variables for different environments without changing configurations.

Command Line Flags

*.auto.tfvars Files

*.tfvars Files

Environment Variables

Variable Block Defaults



Higher
Priority

Lower
Priority

LECTURE

Output Block



Accessing New Resources



```
resource "azurerm_resource_group" "prd" {
  name     = "example-resources"
  location = "West Europe"
}

resource "azurerm_virtual_network" "dv" {
  name                = "example-network"
  resource_group_name = "azure"
  location            = "west"
  address_space       = ["10.0.0.0/16"]
}
```



Write
Code



Provision
Resources

New Resources



Virtual Machine



Kubernetes Service



CosmosDB



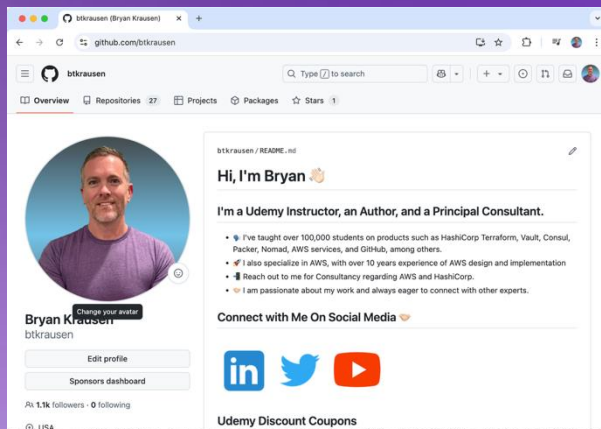
Storage Account



Image Template



Entra Domain



Output Information to
Access Resources



<https://btk.me/btk>



Output Block

An **output** block in Terraform is used to display key information about your infrastructure after a deployment, such as IP addresses, DNS names, or resource IDs. Outputs make it easy to retrieve and share important details without manually looking them up in the cloud provider's console.

They're also useful for passing data between modules or automating workflows.



Output Block

An output block displays important information about your infrastructure, like resource IDs or URLs, after deployment.

- **Display Critical Information**

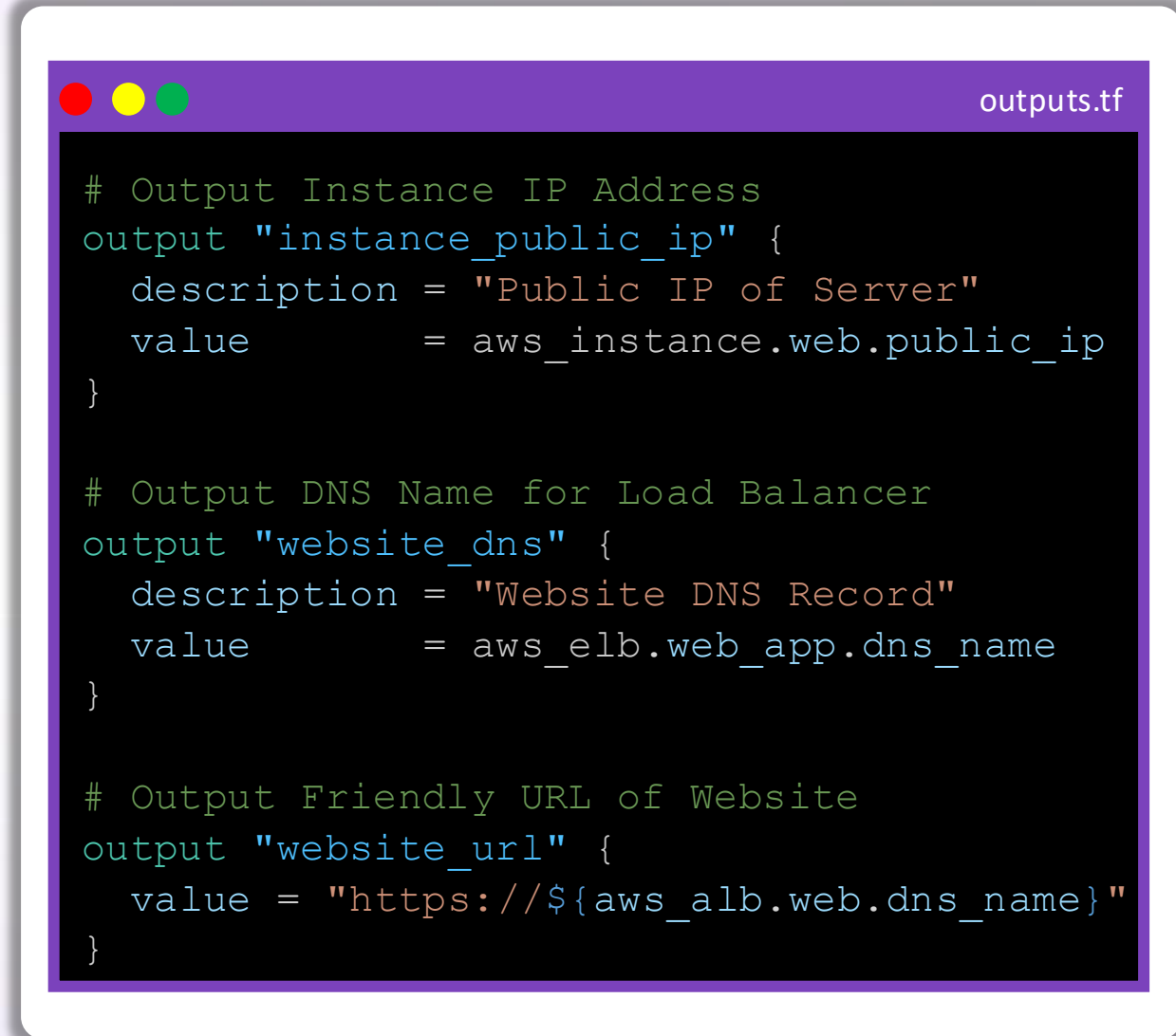
Outputs make it easy to retrieve important details like IP addresses, DNS names, or resource IDs after deployment.

- **Enable Module Integration**

Outputs can pass data between Terraform modules, simplifying workflows and making configs reusable.

- **Improve Workflow Efficiency**

Outputs automate the retrieval of key values, reducing the need for manual lookups in provider consoles.



```
# Output Instance IP Address
output "instance_public_ip" {
  description = "Public IP of Server"
  value       = aws_instance.web.public_ip
}

# Output DNS Name for Load Balancer
output "website_dns" {
  description = "Website DNS Record"
  value       = aws_elb.web_app.dns_name
}

# Output Friendly URL of Website
output "website_url" {
  value = "https://${aws_alb.web.dns_name}"
}
```

Breaking Down the Output Block



outputs.tf

```
# Output Instance IP Address
output "instance_public_ip" {
  description = "Public IP of Web server"
  value       = aws_instance.web.public_ip
}

# Output DNS Name for Load Balancer
output "website_dns" {
  description = "Website DNS Record"
  value       = aws_elb.web_app.dns_name
}

# Output Friendly URL of Website
output "website_url" {
  value = "https://${aws_alb.web.dns_name}"
}
```

The name of the output – user-defined string

The value of the output

Another output block – a more advanced value combining the value of a resource within a string

LECTURE

Terraform Block



Imagine This Real-World Scenario...



Your Workstation

```
resource "azurerm_resource_group" "prd" {
  name      = "example-resources"
  location  = "West Europe"
}

resource "azurerm_virtual_network" "dv" {
  name                = "example-network"
  resource_group_name = "azure"
  location            = "west"
  address_space       = ["10.0.0.0/16"]
}

resource "azurerm_mssql_database" "db1" {
  name            = "example-db"
  server_id       = "server_db"
  collation       = "sql_general"
}
```



Terraform Version
1.10



Commit
Code



Git
Repo



Pull
Code

```
resource "azurerm_resource_group" "prd" {
  name      = "example-resources"
  location  = "West Europe"
}

resource "azurerm_virtual_network" "dv" {
  name                = "example-network"
  resource_group_name = "azure"
  location            = "west"
  address_space       = ["10.0.0.0/16"]
}

resource "azurerm_mssql_database" "db1" {
  name            = "example-db"
  server_id       = "server_db"
  collation       = "sql_general"
}
```

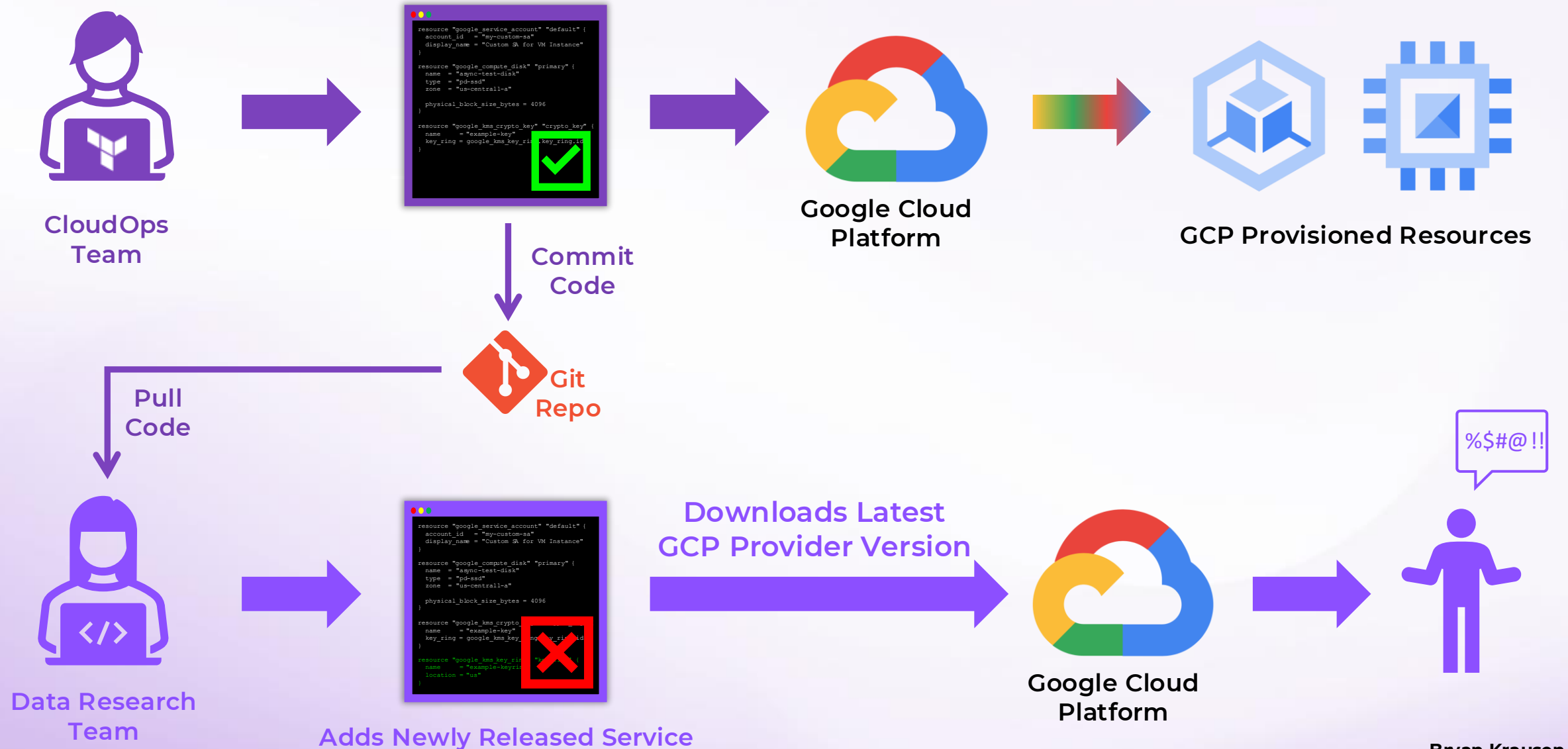


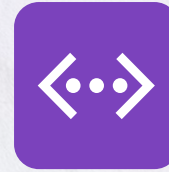
Terraform Version
1.11





The Risk of Mismatched Provider Versions

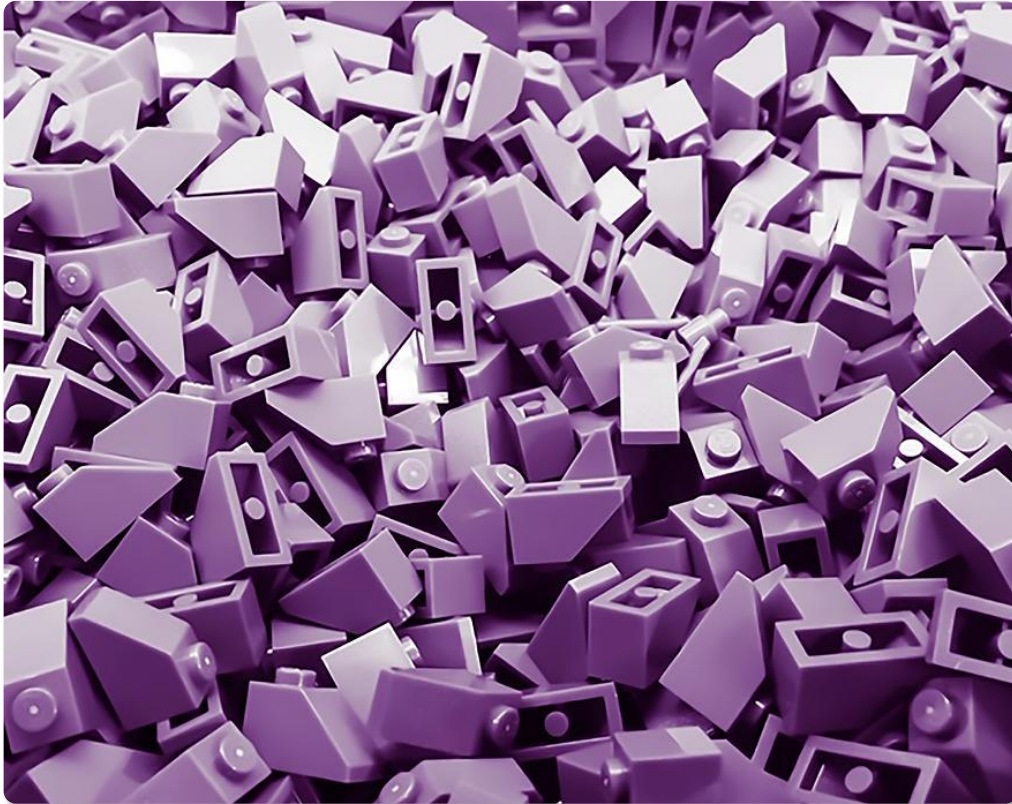




Terraform Block

The **terraform** block sets global configurations for your Terraform project, such as required provider versions, Terraform version constraints, and backend settings for state management. It ensures consistency across teams and environments by enforcing specific versions and configurations.

Using the **terraform** block helps avoid errors and provides a solid foundation for managing infrastructure effectively.





Terraform Block

The `terraform` block ensures your Terraform setup is consistent, and scalable across teams and environments.



Version Control

Specifies the required Terraform and provider versions to avoid compatibility issues



Backend Configuration

Configures where Terraform state is stored to ensure secure and reliable state management



Provider Requirements

Defines the providers and the versions needed for your infrastructure

```
terraform {  
  required_version = "~> 1.10.0"  
  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
  
  backend "s3" {  
    bucket = "prd-terraform-east-2"  
    key    = "state/terraform.tfstate"  
    region = "us-east-2"  
  }  
}
```

Breaking Down the Terraform Block



```
terraform {  
  required_version = "~> 1.10.0"  
  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "5.87.0"  
    }  
  }  
  
  backend "s3" {  
    bucket = "prd-terraform-east-2"  
    key = "state/terraform.tfstate"  
    region = "us-east-2"  
  }  
}
```

Ensures Terraform version 1.10.x is used to run this configuration

Specify a required provider and require version 5.87.0

The backend configuration of where you want to store Terraform state

Version Constraints

```
terraform {
  required_version = "1.9.8"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "5.77.0"
    }
  }
}
```

- ✦ Only Terraform v1.9.8 can be used to run this configuration
- ✦ Installs/uses AWS provider version 5.77.0

```
terraform {
  required_version = ">= 1.9.8"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = ">= 5.77.0"
    }
  }
}
```

- ✦ Terraform v1.9.8 or greater can be used to run this configuration
- ✦ Installs/uses AWS provider version greater or equal to 5.77.0

```
terraform {
  required_version = "~> 1.11.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.87.0"
    }
  }
}
```

- ✦ Terraform v1.11.x or greater can be used to run this configuration
- ✦ Installs/uses AWS provider version greater or equal to 5.87.x

~ allows only the right-most version component to increment